Fundamentos de Sistemas Operacionais

Deadlock

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

Campinas, 2s/2026



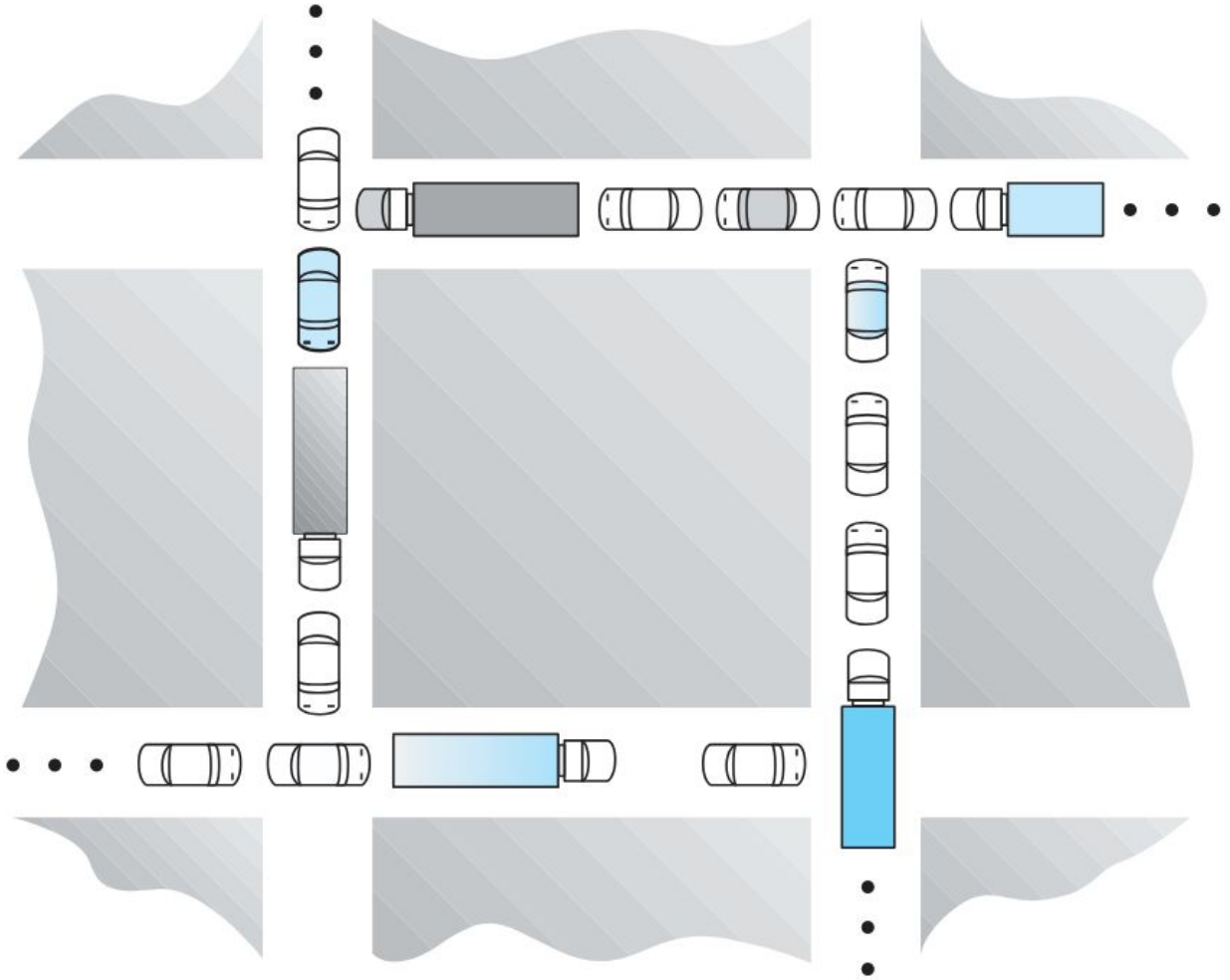
Nesta aula…

- Fenômeno resultantes da má coordenação do uso de recursos do sistema: **deadlock**
- Conceitos chave
  - **Alocação de recursos**
  - **Espera circular**
  - **Deadlock**





Deadlock







Modelo do Sistema

- Sistema consiste de recursos
- Recursos do tipo $R_1, R_2, \ldots, R_m$
  - *Ciclos de CPU,espaço de memória, dispositivos E/S*
- Cada recurso R_i tem $W_i$instâncias.
- Cada processo utiliza um recurso nessa ordem:
  - **request:** bloqueia o recurso, ou espera para que possa bloquear
  - **use:** usa o recurso
  - **release:** libera o bloqueio do recurso para que outro processo possa usar





Caracterização de Deadlock

Para um deadlock ocorrer, as quatro circunstâncias precisam ocorrer

- **Exclusão mútua:** apenas uma thread pode usar o recurso
- **Retenção e espera:** a thread deve estar de posse de pelo menos um recurso e esperando para adquirir recursos adicionais que estejam no momento sendo retidos por outros processos.
- **Inexistência de preempção:** Recursos não podem ser interceptados, apenas liberados voluntariamente pelo processo que o estiver retendo
- **Espera circular:** existe um conjunto $\{T_0, T_1, \ldots, T_n\}$ ode threads esperando de modo que T_0 espera um recurso possuído por T_1, T_1 espera um recurso possuído por T_2, …, T_{n-1} espera um recurso de T_n, e T_n espera um recurso de T_0.





Grafo de Alocação de Recursos

Conjunto de vértices V e arestas E

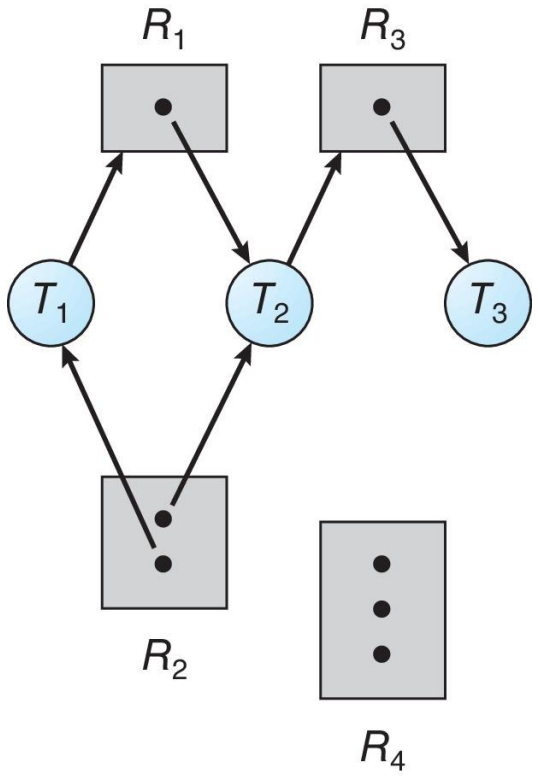
- V é dividido em dois tipos:
  - $T = \{T_1, T_2, \ldots, T_n\}$, conjunto de todas as threads do sistema.
  - $R = \{R_1, R_2, \ldots, R_m\}$, conjunto de todos os tipos de recurso do sistema
- **Aresta de solicitação** – aresta $T_i \rightarrow R_j$
- **Arestas de atribuição** – aresta $R_j \rightarrow T_i$





Exemplo de Alocação de Recursos

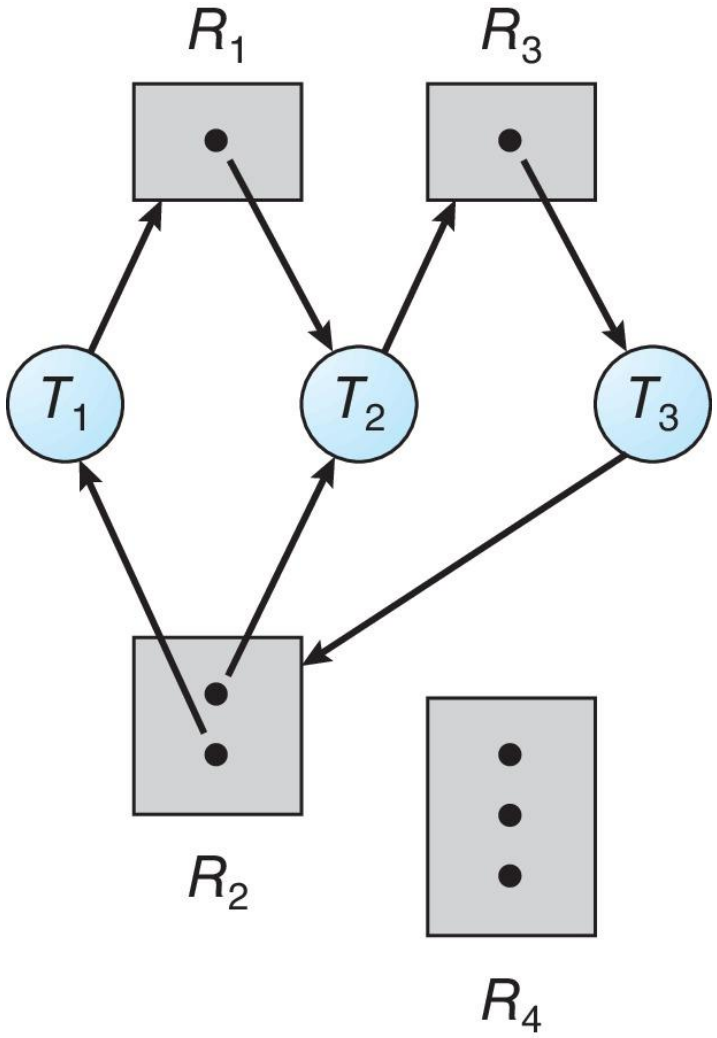
- Uma instância de R_1
- Duas instâncias de R_2
- Uma instância de R_3
- Três instâncias de R_4
- T_1 retêm uma instância de R_2 e espera por uma instância de R_1
- T_2 retêm uma instância de R_1,uma instância de R_2, e espera por uma instância de R_3
- T_3 retêm uma instância de R_3





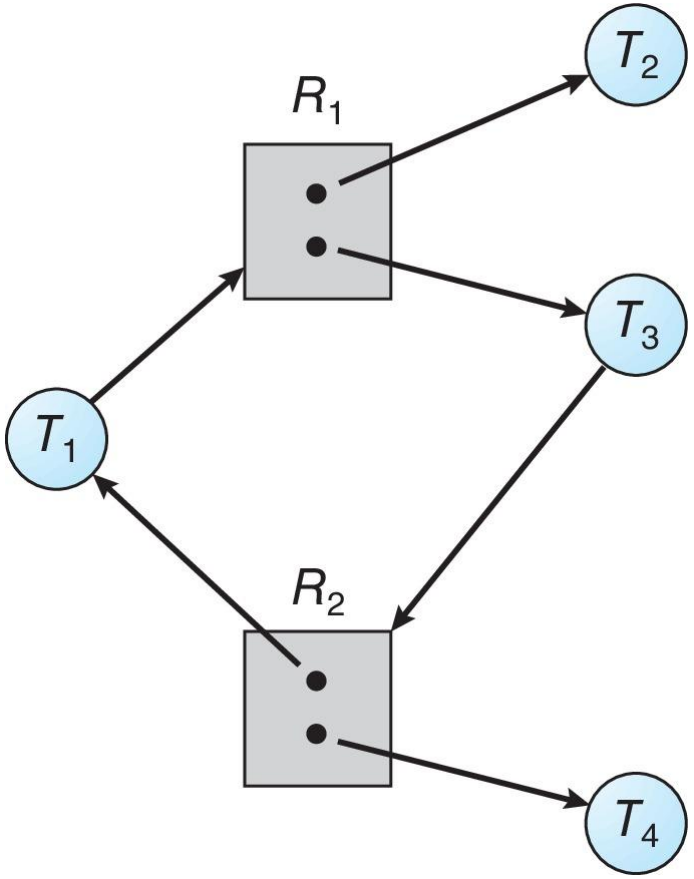


Alocação de recursos com Deadlock





Alocação de recursos em ciclo sem Deadlock





Simplificando…

- Se o grafo não contém ciclos ⇒ não há deadlock
- Se o grafo contem ciclos ⇒
  - Se há uma única instância por tipo de recurso, **deadlock**
  - Se há mais de uma instância por tipo de recurso, *pode haver* deadlock





Métodos de lidar com Deadlocks

- Garantir que o sistema **nunca** entra em estado de deadlock:
  - Previnir deadlocks
  - Evitar deadlocks
- Permitir que o sistema entre em deadlock e então recuperar o estado sem deadlock
- Ignorar o problema e fingir que deadlocks não existem 👍





Prevenção de Deadlocks

Invalidar um dos quatro requisitos para deadlocks:

- **Exclusão Mútua** – não necessária para recursos compartilháveis (e.g., arquivos read-only); deve se manter para arquivos não compartilháveis
- **Retenção e Espera** – devemos garantir que sempre que uma thread requisita um recurso, ela não pode reter outros recursos
  - Requer que threads requisitem e sejam alocadas todos os seus recursos antes de começar a execução ou permitir requisitar apenas recursos que estão livres
  - Pouca utilização de recursos; ainda é possível **starvation**





Prevenção de deadlocks

- **Inexistência de Preempção**:
  - Se um processo retendo alguns recursos requisita outro recurso que não pode ser imediatamente alocado, todos os recursos atualmente retidos são liberados
  - Recursos liberados são adicionados à lista de espera do processo
  - Thread volta à CPU apenas quando puder recuperar seus recursos antigos, além dos novos que estava requisitando
  - Comumente usado para recursos facilmente removidos (registradores da CPU)
- **Espera Circular:**
  - Cria uma ordem para todos os recursos, e ordena que cada thread requisite recursos em ordem





Impedimento de Deadlocks

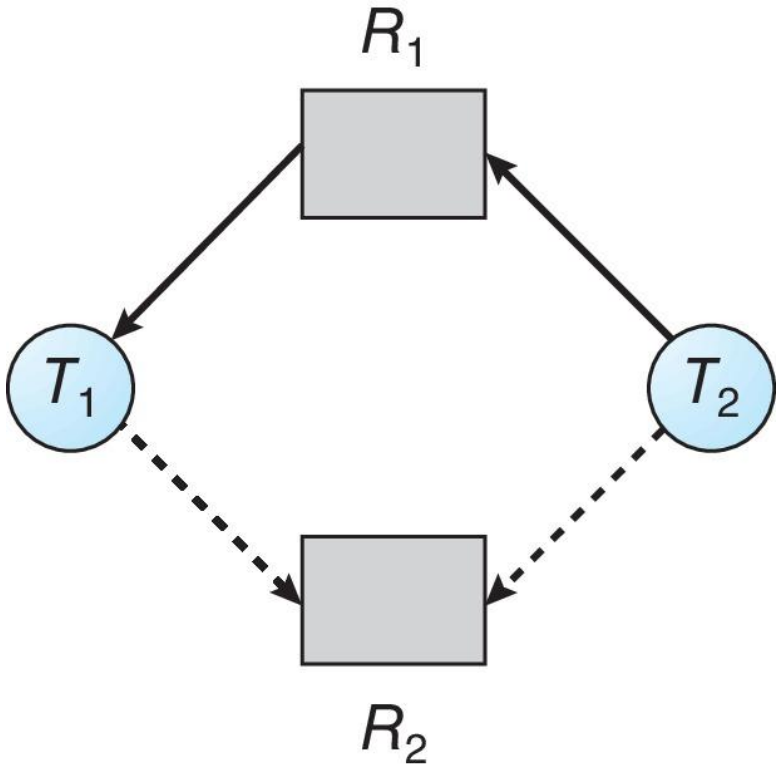
Requer que o sistema possua informações ***a priori*** a mais

- Requer que cada thread declare o **máximo número** de recursos para cada tipo que pode precisar
- O algoritmo que impede deadlocks examina dinamicamente o estado de alocação dos recursos para garantir que nunca há espera circular
- Um *estado* de alocação de recursos é definido pelo número de recursos disponíveis e alocados, e o máximo de demandas dos processos





Grafo de alocação de recursos



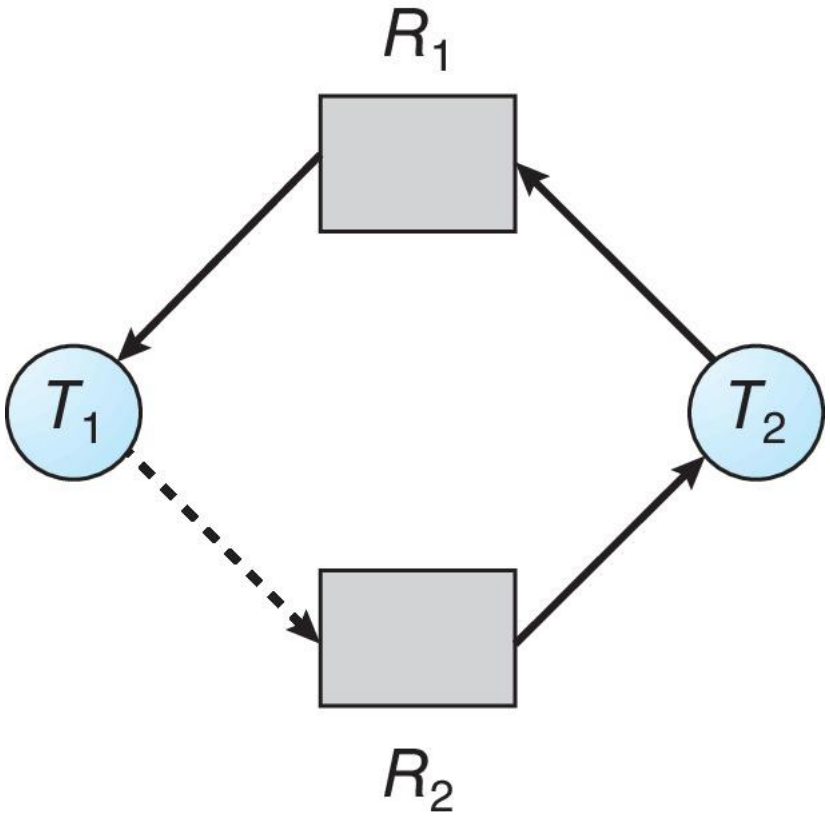
Solicitação

Requisição





Grafo de alocação em estado inseguro







Grafo de alocação

- Suponha que a thread T_i requisita o recurso R_j
- A requisição só pode ser garantida se converter uma aresta de requisição para uma aresta de solicitação não gera formação de um ciclo





Detecção de Deadlock

- Permitir ao sistema entrar em deadlock
- Executar um algoritmo de detecção
- Recuperar o estado de execução





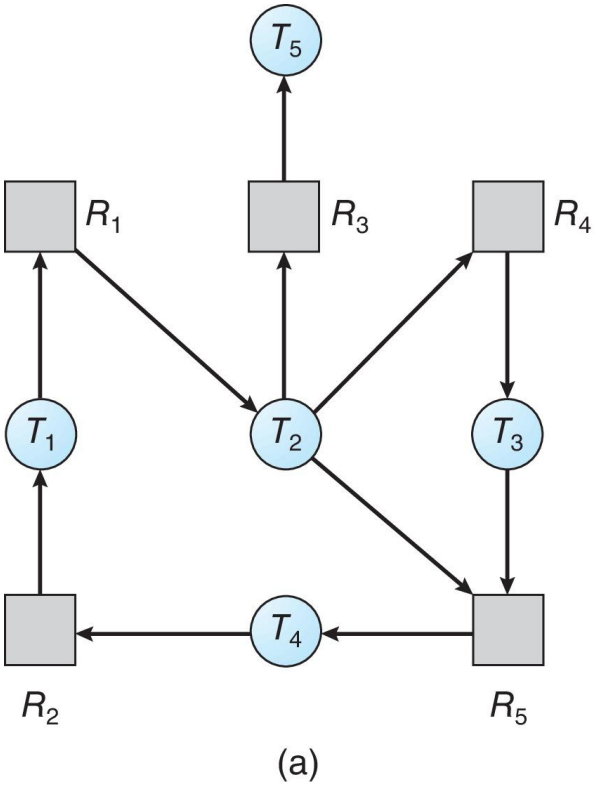
Detecção de Deadlock

- Manter um grafo **de espera (wait-for)**
  - Vértices são processos
  - $T_i \rightarrow T_j$ se T_i está esperando por T_j
- Periodicamente invoca um algoritmo que busca por ciclos no grafo. Se há ciclo, existe deadlock
- Um algoritmo que detecta um ciclo no grafo requer n^2 operações, onde **n** é o número de vértices no grafo

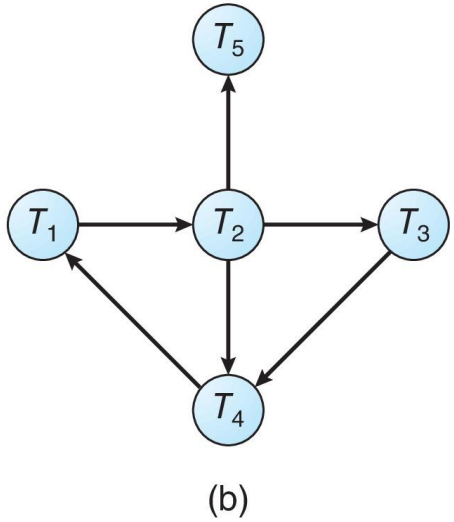




Grafo de Alocação e Grafo de Espera



Grafo de Alocação de Recursos



Grafo wait-for correspondente





Recuperação do Deadlock - Encerrar Processos

- Encerrar todas as threads em deadlock
- Encerrar um processo por vez até que o ciclo seja eliminado
- Em que ordem devemos encerrar?
  1. Prioridade da thread
  2. Por quanto tempo a thread computou, e quando tempo falta?
  3. Recursos que a thread usou
  4. Recursos que a thread precisa para concluir
  5. Quantas threads precisam ser encerradas
  6. …

